

Adaptive Security Architecture for Protecting a Private Source-Code Repository

AWS Reference Architecture — Architecture Explanation and Request / Security Flow

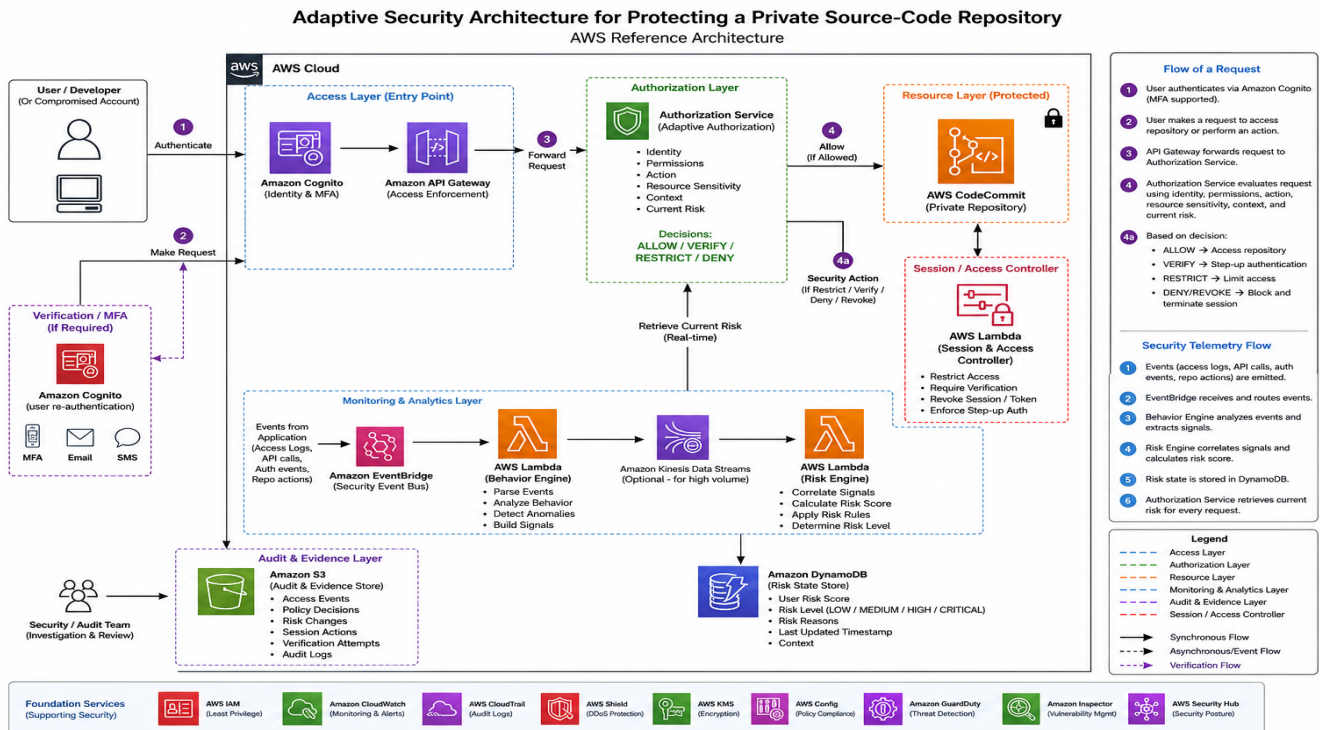


Figure 1. AWS-oriented reference architecture showing authentication, adaptive authorization, behavioral monitoring, risk evaluation, session control, and audit evidence.

1. Executive Summary

This architecture is designed to protect a private source-code repository when a legitimate developer account or session is compromised. The central idea is that successful authentication must not automatically provide unlimited trust. A stolen, valid session should still be constrained by least privilege, continuously evaluated behavior, resource sensitivity, and adaptive authorization.

The security model is: **Authenticate** → **Authorize** → **Monitor** → **Analyze Behavior** → **Calculate Risk** → **Adapt Access** → **Audit**.

2. Problem Being Solved

Traditional authentication answers “**Who are you?**” and authorization answers “**What are you allowed to access?**”. The major weakness is that a valid credential can make a malicious actor look like the legitimate user.

Example: Arun normally works on Project-A. An attacker steals Arun's valid session. The attacker can authenticate successfully and may attempt to enumerate repositories, access Project-B, read sensitive configuration, clone a repository, or download large amounts of source code.

The architecture therefore adds a third question: “**Given the user's current behavior and context, should this action be allowed right now?**”

3. Core Security Model

Stage	Purpose
Least Privilege	Limit what each identity can access before an attack occurs.
Authentication	Verify the identity/session.
Authorization	Check permissions for the requested action and resource.
Behavior Monitoring	Observe access patterns, downloads, repository actions, and context.
Risk Evaluation	Determine how suspicious or dangerous the current activity is.
Adaptive Access	Change the effective decision according to current risk.
Session Control	Restrict, require verification, or revoke access.
Audit / Evidence	Preserve security events and decisions for investigation.

4. AWS Architecture Components

AWS / Logical Component	Responsibility
Amazon Cognito	Identity, authentication and MFA / step-up authentication.
Amazon API Gateway	Controlled application entry point and access-enforcement boundary.
Authorization Service	Custom service that evaluates identity, permissions, action, resource sensitivity, context and current risk.
AWS CodeCommit / Private Git Repository	Protected source-code resource. Repository technology can remain abstract if a different private Git platform is used.
Amazon EventBridge	Routes security and application events to downstream analysis.
AWS Lambda — Behavior Engine	Processes events, detects unusual patterns and creates behavioral signals.
AWS Lambda — Risk Engine	Combines signals and calculates current risk state.
Amazon DynamoDB	Stores shared current risk state so authorization decisions can use fresh, centralized risk information.
Session / Access Controller	Executes security actions such as restrict, verify, revoke, or step-up authentication.
Amazon S3	Stores audit and security evidence such as access events, policy decisions and session actions.

5. Normal Request Flow

- 1. Authenticate:** The developer authenticates through Amazon Cognito. MFA can be required according to the security policy.
- 2. Make request:** The developer requests an action such as reading a file, downloading a file, cloning a repository, or accessing a branch.
- 3. API Gateway:** The request enters through the controlled API boundary.
- 4. Build security context:** The access layer provides identity, role, requested action, resource, repository, session and relevant context to authorization.
- 5. Authorization decision:** The Authorization Service checks permissions, resource sensitivity and the user's current risk state.
- 6. Allow:** If policy permits the action, the request proceeds to the private repository.
- 7. Emit telemetry:** The access/repository action produces a security event for monitoring and analysis.

6. Suspicious Request Flow

- 1. Valid session:** The attacker uses Arun's stolen session. Authentication succeeds because the credentials are valid.
- 2. Unusual behavior:** The attacker accesses many files rapidly, enumerates repositories, changes normal access patterns, or attempts sensitive downloads.
- 3. Event collection:** These actions produce telemetry that is routed through EventBridge.
- 4. Behavior analysis:** The Behavior Engine compares activity with the user's expected behavioral baseline and extracts signals.
- 5. Risk calculation:** The Risk Engine combines signals into a current risk state such as LOW, MEDIUM, HIGH or CRITICAL.
- 6. Adaptive authorization:** The next sensitive request is evaluated using the current risk state.
- 7. Security action:** Depending on policy, the system can ALLOW, RESTRICT, VERIFY, DENY or REVOKE.

7. Example Attack Scenario

Scenario: An attacker obtains Arun's valid session and requests DOWNLOAD for a sensitive security configuration file.

Check	Result
Authentication	Valid session — authentication succeeds.
Identity	Arun.
Permission	Arun may normally have access to the repository.
Resource sensitivity	High — the requested security configuration is more sensitive than ordinary source files.
Current behavior	Recent unusual repository access and bulk file reads have increased risk.
Risk state	HIGH.
Policy decision	VERIFY rather than immediately allow the download.
If verification succeeds	The policy may permit the action, depending on sensitivity.
If verification fails	Restrict or revoke the session and record the event.

8. Risk Is More Than a Download Count

The architecture should not depend on a single threshold such as “more than 100 files means an attack.” A realistic risk calculation can consider multiple dimensions: unusual repository access, unusual file volume, new device, new location, unusual time, sensitive resources, failed verification, permission changes, and the sequence of actions.

For example, a new device alone may produce only a small risk increase. A new device combined with a new location, repository enumeration, and mass downloads can produce a much stronger risk signal.

9. Resource Sensitivity

User risk and resource risk are separate concepts. A low-risk user accessing a highly sensitive signing key or production credential should not automatically receive the same treatment as a low-risk user reading an ordinary source file.

The authorization decision therefore considers at least: **user risk + resource sensitivity + requested action + context + permissions**.

10. Synchronous vs. Asynchronous Security Paths

Synchronous request path: Developer → Authentication → API Gateway → Authorization Service → Repository. This path must remain fast because it is executed for access requests.

Asynchronous telemetry path: Repository / access events → EventBridge → Behavior Engine → Risk Engine → DynamoDB Risk State. The behavior analysis is intentionally separated from every request so expensive analysis does not have to block every normal request.

The Authorization Service reads the current risk state when making an access decision. Highly sensitive operations can require fresher risk information or additional verification.

11. Trust Boundaries

The developer/attacker side must be treated as untrusted. API Gateway and the access layer form an exposed enforcement boundary. The repository, authorization logic, risk state and audit evidence are protected assets. There must not be an alternate direct path that allows a caller to bypass the access-enforcement and authorization layers.

12. Least Privilege

Least privilege is the first layer of defense. A developer should receive only the repositories, branches, actions and resources needed for their job. A compromised account therefore starts from a limited capability rather than from organization-wide access.

13. Behavioral Baselines

The Behavior Engine learns what is normal for a particular user, role, repository and context. Examples include typical access volume, repositories normally used, normal working times, common actions and usual devices or environments.

Baseline updates should be controlled. If every attacker action is immediately treated as normal, the attacker can gradually poison the baseline. Updates should therefore use confidence, validation and controlled adaptation.

14. Defense in Depth

No single component is expected to stop every compromise. The architecture deliberately layers controls:

- Least privilege limits initial capability.
- Authentication protects identity.
- API Gateway provides a controlled entry point.
- Authorization checks what the identity may do.
- Behavior monitoring detects deviations.
- Risk evaluation converts behavior into a security state.
- Adaptive authorization changes the decision as conditions change.
- Session control can restrict, verify or revoke access.
- S3 audit evidence supports investigation and accountability.

15. Failure and Attack Considerations

Risk Engine unavailable: Do not automatically fail open for highly sensitive actions. The policy should define different fallback behavior by resource sensitivity.

Policy Engine compromised: Protect policy configuration with strong administrative controls, least privilege, restricted management access, auditing and change monitoring.

Slow exfiltration: Do not rely only on volume thresholds. Analyze sequences, timing, resource sensitivity and multiple behavioral dimensions.

Baseline poisoning: Use controlled baseline updates and confidence before accepting new behavior as normal.

Stolen valid JWT: A valid token proves authentication but does not guarantee unrestricted authorization.

16. Security Evidence and Audit

Important events should be recorded, including login events, repository access, file downloads, clone operations, permission changes, token refreshes, verification attempts, risk changes, policy decisions, denied actions and session revocations. The audit layer should be access-controlled and designed to preserve reliable evidence for security investigation.

17. One-Page Mental Model

Compromised Account → Least Privilege → Behavior Monitoring → Risk Evaluation → Adaptive Authorization → Restrict / Verify / Revoke → Audit

The most important architectural principle is that **authentication is not permanent trust**. A user can remain authenticated while their effective access is reduced because their current behavior has become risky.

18. Expected Security Outcome

The architecture does not claim that credentials can never be stolen. Instead, it reduces the blast radius of a compromised identity. A stolen session should not automatically provide unrestricted repository access, and suspicious behavior should cause the system to become progressively more defensive.

19. Important Scope Note

AWS services in this diagram are implementation building blocks. The adaptive-security behavior itself is a system design, not a feature automatically provided by a single AWS service. The Authorization Service, Behavior Engine, Risk Engine and Session/Access Controller represent application-level security logic that would need to be designed and implemented.